Assignment Submission

Course Code: SE113

Course Name: Introduction to Software Engineering

Topic Name: SDLC Models Discussion

Submitted To

Name: Mr. Nasim Mahmud Likhon

Designation: Lecturer

Department: Software Engineering

Daffodil International University

Submitted By

Name: Mohammad Atikul Hoque Showrav

ID: 261-35-272

Section: F

Semester: Spring 26, 261

Department: Software Engineering

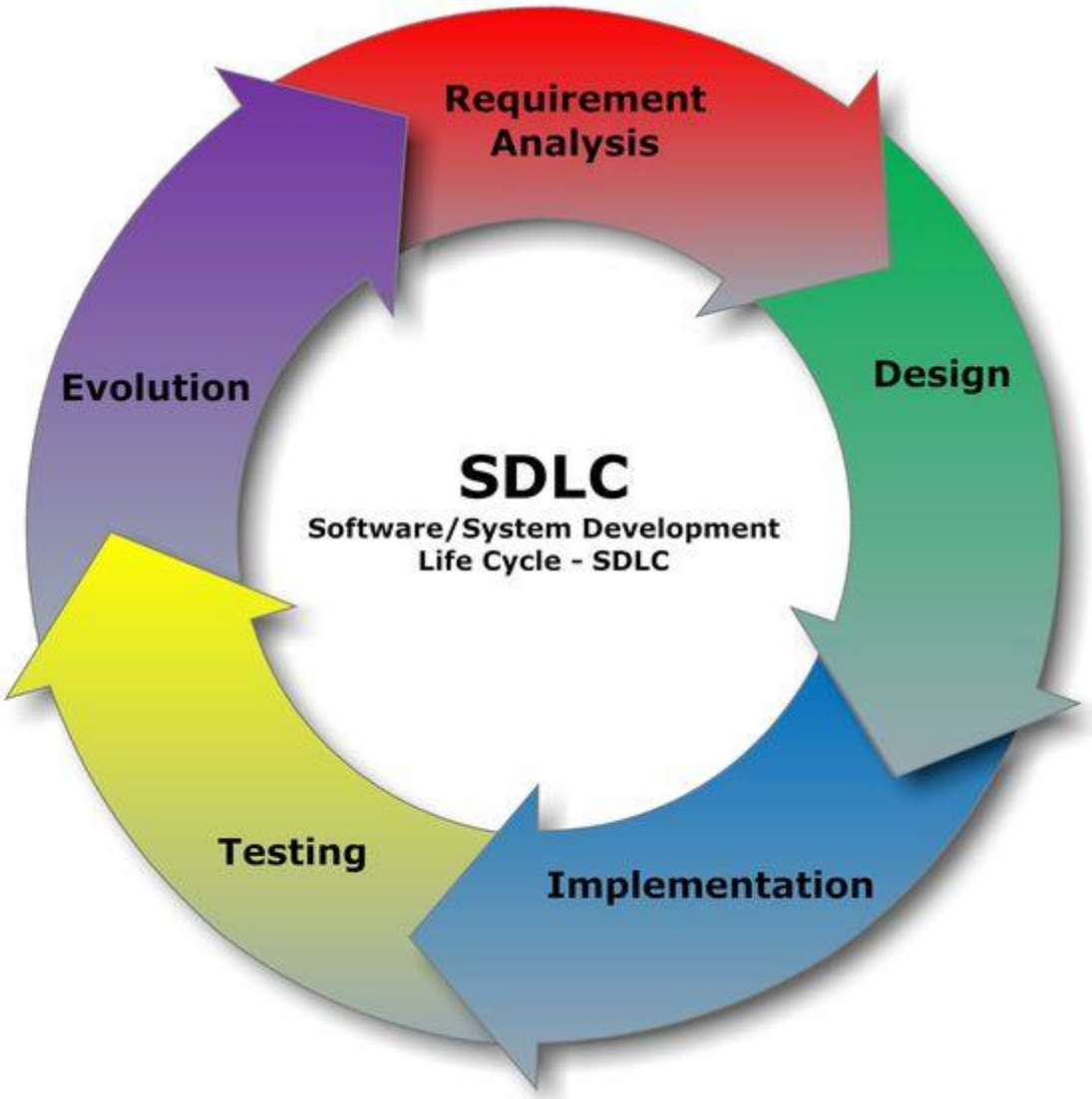
Daffodil International University

Submission Date: 27/01/2026

www.diucoverpage.com

Software Development Life Cycle (SDLC)

The Software Development Life Cycle (SDLC) is a structured framework in software engineering that defines the stages of building, deploying, and maintaining software. It ensures quality, efficiency, and risk control by following systematic phases such as requirements gathering, design, coding, testing, deployment, and maintenance.



Key Phases of SDLC

- **Requirement Analysis: Collecting and documenting user needs and system requirements.**
- **Feasibility Study: Evaluating economic, technical, legal, and operational viability.**
- **System Design: Creating high-level and detailed designs (architecture, database, UI).**
- **Implementation (Coding): Developers write code based on design specifications.**
- **Testing: Verifying functionality, performance, and security; fixing defects.**
- **Deployment: Releasing the software to production or end-users.**
- **Maintenance: Updating, fixing, and enhancing the system post-deployment.**

Popular SDLC Models

According to Assignment work , there are some popular SDLC models. I am doing discussion on 4 models. They are :

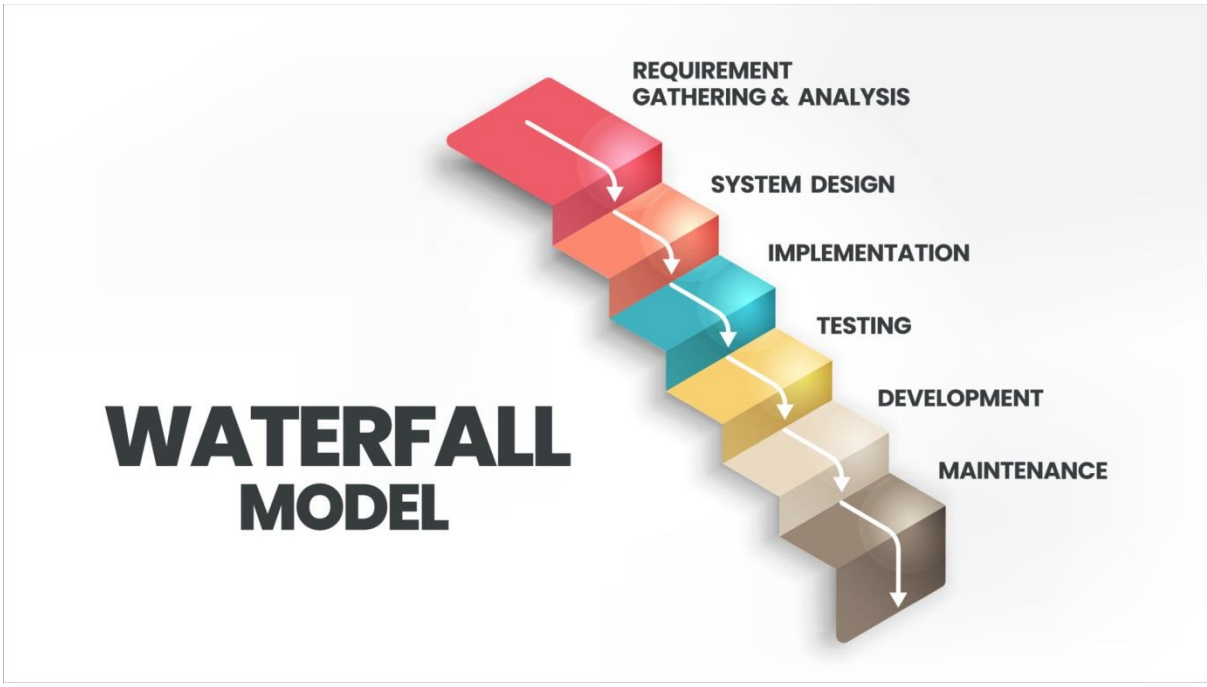
- WaterFall Model
- V Model
- Spiral Model
- Prototype Model

Table on These 4 Models

Model	Approach	Best Use Case	Limitations
WaterFall	Linear, sequential	Small projects with clear requirements	Rigid, hard to adapt to changes
V-Model	Verification & validation at each stage	Safety-critical systems	Expensive, requires detailed documentation
Spiral	Iterative + risk analysis	Large, complex projects with high risk	Costly, requires strong risk management
Prototype	Build a working model (prototype) of the system early, refine it through user feedback until final system is developed	Projects where requirements are unclear or evolving; systems needing strong user interaction (UI-heavy apps, customer-facing tools)	Can lead to scope creep; users may confuse prototype with final product; repeated iterations may increase cost/time

Waterfall Model

The Waterfall Model is a linear and sequential SDLC approach where each phase (requirements, design, implementation, testing, deployment, maintenance) must be completed before moving to the next. It is best suited for projects with well-defined, stable requirements.



Scenario

A government agency wants a payroll management system where requirements are fixed, such as salary calculation, tax deduction, and leave tracking. Analysts gather and document all requirements upfront, and designers create the system architecture along with the database schema. Developers then code the modules strictly following the design, after which testers validate the functionality against the documented requirements. Once testing is successful, the system is deployed and employees begin using it. Finally, the maintenance team handles bug fixes and updates. The entire project follows a strict step-by-step flow with no overlap, making it a classic example of the Waterfall Model.

Why this Scenario suits only Waterfall Model, not the other three

Why Waterfall Fits

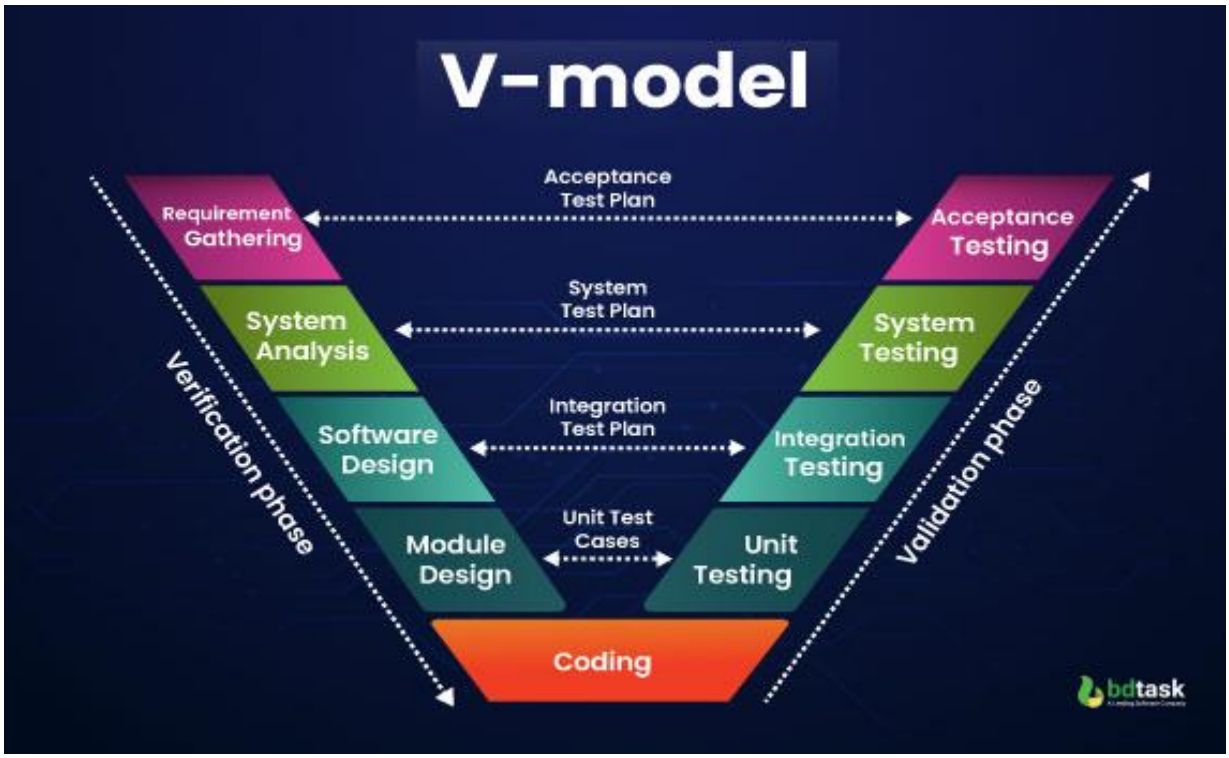
The payroll system has fixed, well-documented requirements (salary calculation, tax deduction, leave tracking). Each phase can be completed in a strict sequence — requirements → design → coding → testing → deployment → maintenance — without needing constant feedback or risk analysis. Government projects also demand formal documentation and predictability, which aligns perfectly with Waterfall's linear approach.

Why other models not fit

The payroll system scenario doesn't fit Spiral, V-Model, or Prototype because Spiral is meant for high-risk, complex projects, while payroll systems are straightforward with low risk; V-Model is best for safety-critical domains requiring rigorous validation, which payroll doesn't demand and Prototype suits projects with unclear or evolving requirements, but here the requirements are fixed, making prototyping unnecessary and inefficient.

V- Model

The V-Model (Verification and Validation Model) is an extension of the Waterfall approach where each development phase has a corresponding testing phase. It emphasizes quality assurance by verifying outputs at every stage and validating them against requirements, making it suitable for safety-critical or highly regulated systems.



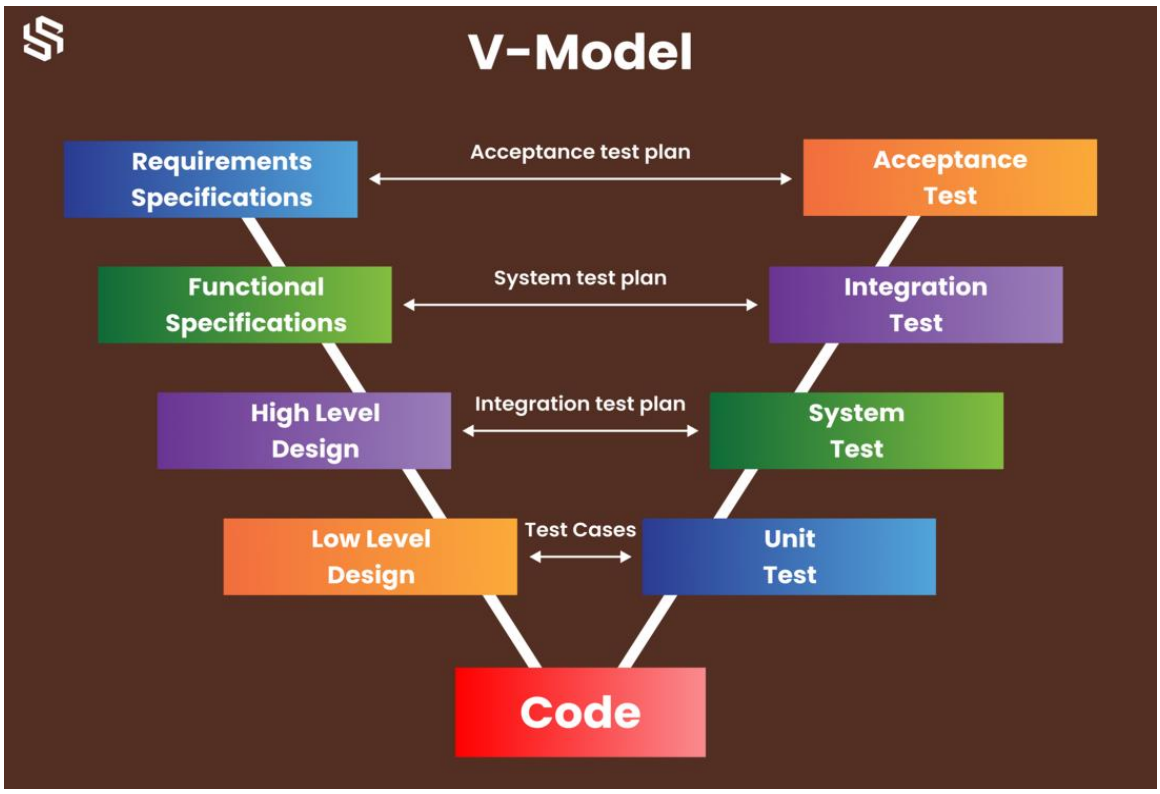
Scenario

A medical equipment company is developing software for a heart rate monitoring device. The requirements are strictly defined by healthcare regulations, so analysts document them in detail. Designers create the system architecture and specifications, which are later verified through design reviews. Developers implement the code following these specifications, while testers prepare validation plans in parallel. As coding progresses, unit tests are conducted to ensure each module works correctly. Integration

testing validates that modules interact as expected. System testing ensures the device meets performance and safety standards. Finally, acceptance testing confirms compliance with medical regulations before deployment. The project follows a strict verification and validation process at each stage, making the V-Model ideal for this scenario.

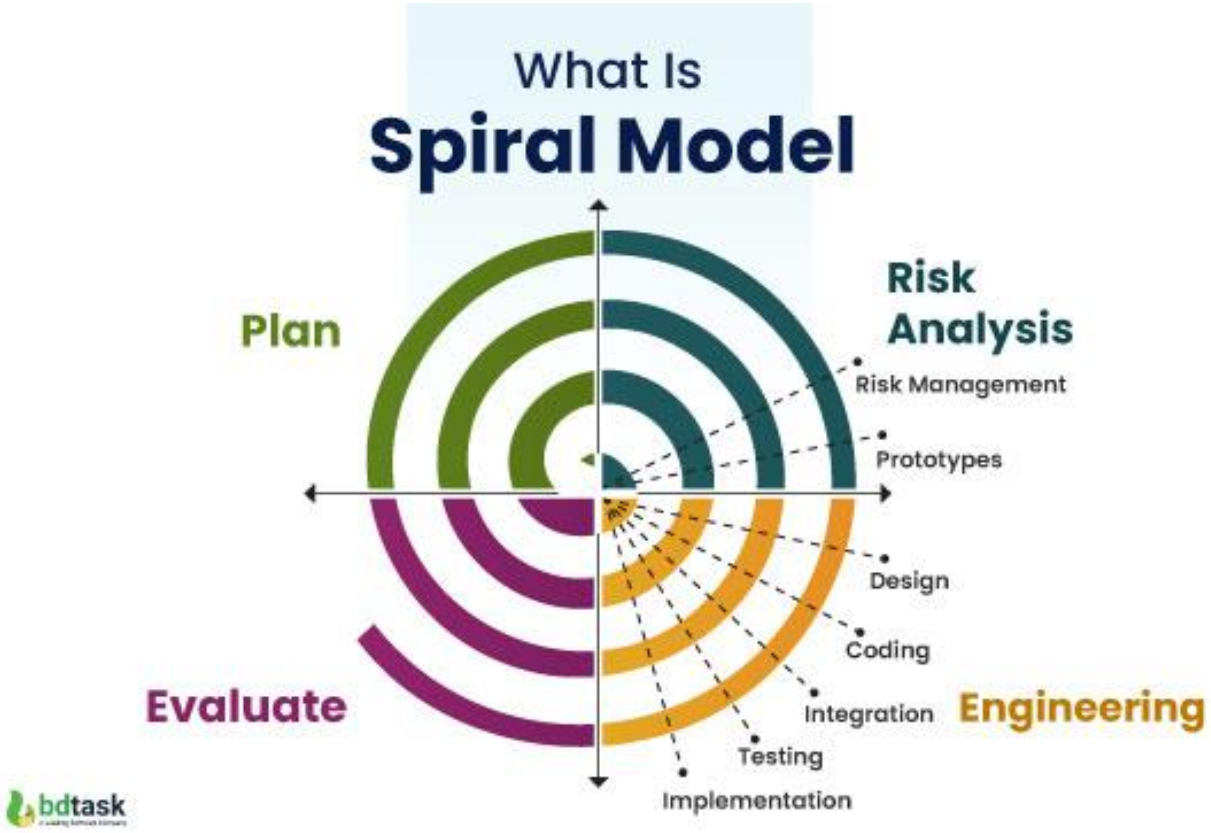
Why other models not fit

The medical device scenario suits the V-Model because it demands strict verification and validation at every stage, but the other three models don't fit well. Waterfall is too rigid since it lacks the parallel testing emphasis needed for safety-critical systems. Spiral focuses on risk analysis and iterative cycles, which adds unnecessary cost and complexity when requirements are already fixed by regulations. Prototype relies on evolving requirements and user feedback, but in this case requirements are clear and must strictly comply with medical standards, making prototyping unsuitable.



Spiral Model

The Spiral Model is an SDLC approach that combines iterative development with systematic risk analysis. It emphasizes repeated cycles (spirals) of planning, prototyping, testing, and evaluation, making it suitable for large, complex, and high-risk projects.



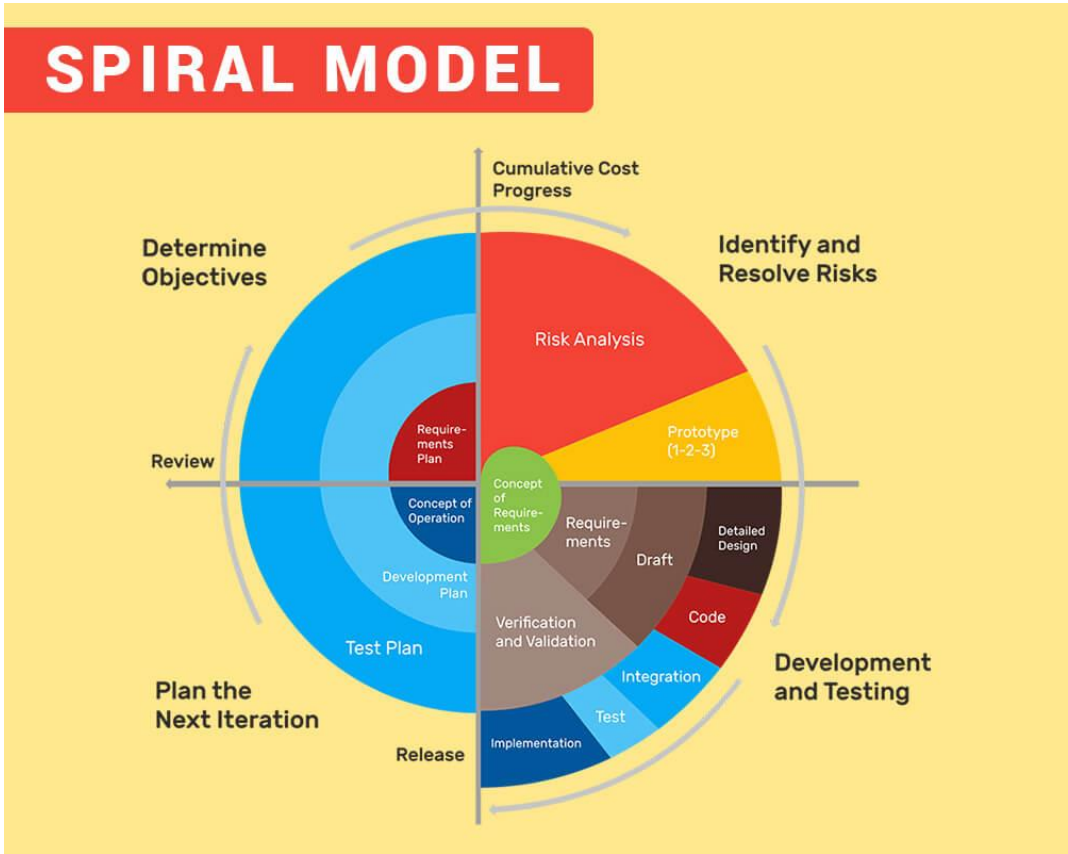
Scenario

A defense contractor is developing a missile control system where safety and reliability are critical. The team begins with planning and identifying potential risks, such as hardware compatibility and security vulnerabilities. A prototype of the control interface is built and tested to gather feedback. Based on results, risks are reassessed and new design improvements are planned. The next spiral includes coding modules and integrating them with hardware, followed by rigorous testing. Each cycle adds more

functionality while continuously analyzing risks. Stakeholders review progress at the end of every spiral to ensure compliance with defense standards. The process continues until the system is fully developed, tested, and deployed. This repeated risk-focused cycle makes the Spiral Model ideal for such a high-risk project.

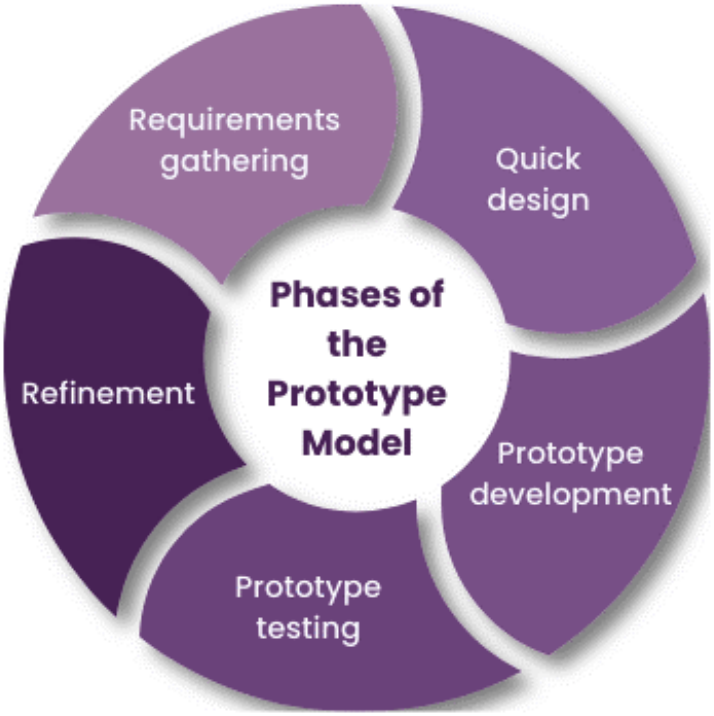
Why other models not fit

The missile control system scenario suits the Spiral Model because it requires repeated risk analysis and iterative refinement, but the other three models don't fit well. **Waterfall** is too rigid since it assumes stable requirements and doesn't address the high risks involved. **V-Model** focuses on verification and validation but is more suited to safety-critical systems with fixed requirements, not projects needing ongoing risk assessment. **Prototype** is designed for unclear or evolving requirements, yet here the requirements are clear but the risks are complex, so prototyping alone cannot handle the risk-driven nature of the project.



Prototype Model

The Prototype Model **is an SDLC approach where a preliminary version of the software (prototype) is built quickly to demonstrate functionality. It allows users to interact with the prototype, provide feedback, and refine requirements until the final system is developed. It is best for projects with** unclear or evolving requirements.



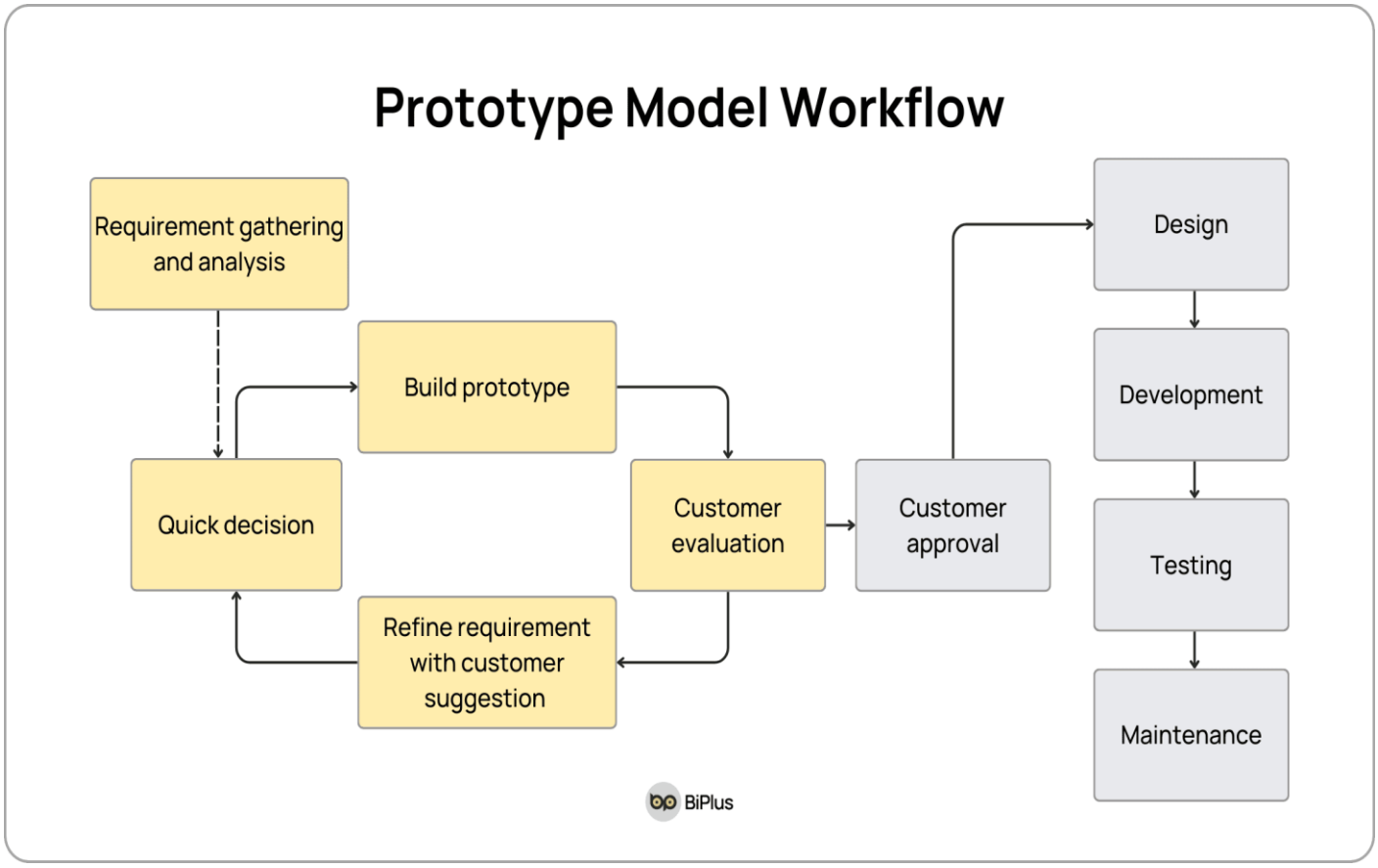
Scenario

A startup wants to develop an online food delivery app but is unsure about the exact features customers will prefer. Developers create a simple prototype showing restaurant listings, order placement, and payment options. Users test the prototype and suggest improvements, such as adding real-time delivery tracking and multiple payment gateways. The team refines the prototype

based on this feedback and builds another version with updated features. Further testing reveals the need for loyalty rewards and push notifications. Each cycle helps clarify requirements and align the app with customer expectations. Once the prototype satisfies users, the final system is developed, tested, and deployed. This iterative refinement makes the Prototype Model ideal for the project.

Why other models not fit

The food delivery app scenario suits the Prototype Model because requirements are unclear and need user feedback, but the other three models don't fit well. **Waterfall** assumes stable requirements and a strict sequential flow, which doesn't work when features are uncertain. **V-Model** is best for safety-critical systems with fixed requirements, not for consumer apps where needs evolve. **Spiral** focuses on risk analysis in large, complex projects, while here the main challenge is clarifying requirements, not managing high risks, so Spiral would add unnecessary cost and complexity.



Thank You